

Práctica 2

CRUD de productos para comercio electrónico

Registro, listado, edición y eliminación con Eloquent, validación y Blade

Laravel 13 · PHP 8.4 · PostgreSQL · Eloquent · Individual · 100 pts

REVISIÓN Viernes 2 de octubre · durante la clase

GET **POST** **PUT** **DELETE**

01 Objetivo y competencias

Al finalizar esta práctica, usted será capaz de implementar las cuatro operaciones básicas sobre una base de datos (crear, leer, actualizar y eliminar: **CRUD**) en una aplicación Laravel 13, persistiendo la información en PostgreSQL mediante Eloquent y validando los datos que ingresa el usuario.

Competencias específicas:

- Diseñar una tabla mediante migraciones y representarla con un modelo Eloquent.
- Poblar la base de datos con datos de prueba usando factories y seeders.
- Implementar un controlador de recursos con sus siete acciones y registrar las rutas con `Route::resource`.
- Validar los datos de los formularios y mostrar los errores al usuario conservando lo que escribió (`old()`, `@error`).
- Construir formularios seguros con `@csrf` y `@method`, y retroalimentar al usuario con mensajes flash.
- Versionar el trabajo con commits atómicos y defender oralmente las decisiones tomadas en el código.

02 Descripción del proyecto

En la Práctica 1 construyó el catálogo de **ByteStore** a partir de un arreglo estático en el controlador. Ahora la tienda necesita un **panel de administración** donde el encargado pueda registrar, consultar, modificar y dar de baja los productos, y esos datos deben quedar guardados en una base de datos real.

Lo que ya se le entrega resuelto en esta guía:

- La configuración de la conexión a PostgreSQL (sección 03).
- La especificación de la tabla y el modelo `Producto` (sección 04).
- Las rutas del recurso (sección 05) y un punto de partida del controlador (sección 06).
- Las reglas de validación y el puente entre formulario y controlador (sección 07).

Lo que usted debe desarrollar (el trabajo independiente):

- La migración, la factory y el seeder de productos.
- Los siete métodos del `ProductoController` con su validación.
- Las vistas del CRUD (listado, formulario de registro, formulario de edición y detalle) con Blade y Tailwind CSS v4.

FLUJO

```
Formulario → POST /productos → ProductoController@store → validación →
Producto::create() → PostgreSQL → redirect + mensaje
```

03 Preparación del entorno

Puede continuar sobre el proyecto `tienda-app` de la Práctica 1 (así reutiliza el layout que ya construyó) o crear uno nuevo con `laravel new tienda-app`. Primero cree la base de datos en PostgreSQL:

```
terminal
# Crear la base de datos (también puede hacerlo desde pgAdmin)
psql -U postgres -c "CREATE DATABASE tienda_app;"
```

Copiar

Luego configure la conexión en el archivo `.env` de la raíz del proyecto:

```
.env
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=tienda_app
DB_USERNAME=postgres
DB_PASSWORD=su_contraseña
```

Copiar

```
terminal
# Verificar la conexión: crea las tablas base de Laravel
php artisan migrate

# Levantar Vite (Tailwind) y el servidor en terminales separadas
npm run dev
php artisan serve
```

Copiar

NOTA

Si `php artisan migrate` responde con el error **could not find driver**, habilite las extensiones `pdo_pgsql` y `pgsql` en su archivo `php.ini` y vuelva a intentarlo. Si se equivocó en la contraseña, el error será **password authentication failed**.

04 Modelo de datos

Genere el modelo junto con su migración, su factory y su seeder en un solo comando:

```
terminal
# -m migración -f factory -s seeder
php artisan make:model Producto -mfs
```

Copiar

Complete la migración `database/migrations/xxxx_create_productos_table.php` para que la tabla `productos` cumpla **exactamente** con la siguiente especificación:

CAMPO	TIPO (MIGRACIÓN)	RESTRICCIONES
id	id()	Clave primaria autoincremental
nombre	string('nombre', 120)	Obligatorio
sku	string('sku', 30)	Obligatorio y único (código interno del producto)
descripcion	text('descripcion')	Opcional: nullable()
categoria	string('categoria', 50)	Obligatorio: Audio, Cómputo, Accesorios o Wearables
precio	decimal('precio', 10, 2)	Obligatorio, en bolivianos, no negativo
stock	unsignedInteger('stock')	Por defecto 0
imagen	string('imagen')	Opcional (URL de la imagen): nullable()
destacado	boolean('destacado')	Por defecto false

CAMPO	TIPO (MIGRACIÓN)	RESTRICCIONES
timestamps	timestamps()	created_at y updated_at

El modelo ya se le entrega. Declara qué campos se pueden asignar de forma masiva y cómo se convierten sus tipos. Cópelo tal cual:

```
app/Models/Producto.php

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Producto extends Model
{
    use HasFactory;

    // Campos que se pueden llenar con create() y update()
    protected $fillable = [
        'nombre', 'sku', 'descripcion', 'categoria',
        'precio', 'stock', 'imagen', 'destacado',
    ];

    // Conversión automática de tipos al leer/escribir
    protected function casts(): array
    {
        return [
            'precio' => 'decimal:2',
            'destacado' => 'boolean',
        ];
    }
}
```

DEFINICIÓN · ASIGNACIÓN MASIVA

Asignación masiva: es crear o actualizar un registro pasando un arreglo completo de datos, por ejemplo `Producto::create($datos)`. Laravel solo acepta los campos listados en `$fillable`; así evita que un usuario malintencionado envíe campos que no deberían modificarse desde un formulario.

Por último, programe la factory con datos realistas usando `fake()` y llame al seeder desde `DatabaseSeeder` para generar **al menos 20 productos**. Compruebe el resultado reconstruyendo la base de datos:

```
terminal

# Borra todas las tablas, vuelve a migrar y ejecuta los seeders
php artisan migrate:fresh --seed
```

05 Rutas del proyecto

Genere el controlador de recursos enlazado al modelo y registre las rutas en `routes/web.php`. Ya están escritas: solo debe copiarlas.

terminal

Copiar

```
php artisan make:controller ProductoController --resource --model=Producto
```

routes/web.php

Copiar

```
<?php

use App\Http\Controllers\ProductoController;
use App\Http\Controllers\TiendaController;
use Illuminate\Support\Facades\Route;

// Catálogo público (Práctica 1)
Route::get('/', [TiendaController::class, 'index'])->name('tienda.index');

// Panel de administración: CRUD de productos (registra 7 rutas)
Route::resource('productos', ProductoController::class);
```

Una sola línea de `Route::resource` registra las siete rutas del CRUD. Verifíquelas con `php artisan route:list --name=productos`:

MÉTODO	URI	NOMBRE	ACCIÓN
GET	/productos	productos.index	index
GET	/productos/create	productos.create	create
POST	/productos	productos.store	store
GET	/productos/{producto}	productos.show	show
GET	/productos/{producto}/edit	productos.edit	edit
PUT	/productos/{producto}	productos.update	update
DELETE	/productos/{producto}	productos.destroy	destroy

06 Controlador de recursos

El controlador generado trae los siete métodos vacíos. Cada uno tiene una responsabilidad concreta:

MÉTODO	RESPONSABILIDAD	RESPUESTA
index	Listar los productos paginados (10 por página), los más recientes primero	view productos.index
create	Mostrar el formulario de registro vacío	view productos.create
store	Validar los datos y crear el registro	redirect + flash
show	Mostrar el detalle de un producto	view productos.show
edit	Mostrar el formulario con los datos actuales cargados	view productos.edit
update	Validar (SKU único, ignorando el propio registro) y actualizar	redirect + flash
destroy	Eliminar el registro	redirect + flash

Como punto de partida se le muestran dos métodos: uno que **lee** y otro que **modifica** la base de datos. Los cinco restantes son su trabajo.

```
app/Http/Controllers/ProductoController.php (fragmento) Copiar

// Leer: consulta paginada y envío a la vista
public function index()
{
    $productos = Producto::latest()->paginate(10);

    return view('productos.index', compact('productos'));
}

// Modificar: Laravel ya encontró el producto por su id (Route Model Binding)
public function destroy(Producto $producto)
{
    $producto->delete();

    return redirect()->route('productos.index')
        ->with('success', 'Producto eliminado correctamente.');
```

DEFINICIÓN · ROUTE MODEL BINDING

Route Model Binding: cuando un método recibe un parámetro tipado como `Producto $producto`, Laravel toma el `{producto}` de la URL, busca el registro con ese id y se lo entrega ya cargado. Si no existe, responde automáticamente con un error 404. Úselo en `show`, `edit`, `update` y `destroy`.

07 Validación y formularios

Nunca guarde en la base de datos lo que llega de un formulario sin validarlo. Tanto `store` como `update` deben aplicar, como mínimo, las siguientes reglas con `$request->validate()` (o, si lo prefiere, con una Form Request):

CAMPO	REGLAS
nombre	required string max:120
sku	required string max:30 unique:productos,sku
descripcion	nullable string max:1000
categoria	required in:Audio,Cómputo,Accesorios,Wearables
precio	required numeric min:0
stock	required integer min:0
imagen	nullable url
destacado	nullable boolean

Al **editar**, la regla `unique` fallaría con el SKU del mismo producto que se está modificando. Indíquele a Laravel que ignore ese registro:

```
ProductoController@update (regla del SKU) Copiar

use Illuminate\Validation\Rule;

'sku' => ['required', 'string', 'max:30',
    Rule::unique('productos', 'sku')->ignore($producto)],
```

NOTA · CASILLA DE VERIFICACIÓN

Un checkbox desmarcado **no se envía** en la petición. Para que destacado se guarde como false al desmarcarlo, asigne su valor explícitamente antes de guardar: `$datos['destacado'] = $request->boolean('destacado');`

El siguiente fragmento muestra únicamente el puente entre el formulario y el controlador. No es la solución: el diseño y el resto de los campos son su trabajo.

```
resources/views/productos/edit.blade.php (esqueleto mínimo) Copiar

{{-- Solo muestra el puente de datos. El diseño lo construye usted. --}}
<form action="{{ route('productos.update', $producto) }}" method="POST">
    @csrf
    @method('PUT')

    <input type="text" name="nombre" value="{{ old('nombre', $producto->nombre) }}">
    @error('nombre')
        <p>{{ $message }}</p>
    @enderror

    <button type="submit">Guardar cambios</button>
</form>

{{-- Eliminar: formulario independiente con confirmación --}}
<form action="{{ route('productos.destroy', $producto) }}" method="POST"
    onsubmit="return confirm('¿Eliminar este producto?')">
    @csrf
    @method('DELETE')
    <button type="submit">Eliminar</button>
</form>
```

ADVERTENCIA

Los navegadores solo envían formularios con GET o POST. Sin `@method('PUT')` o `@method('DELETE')` la petición no llegará a update ni a destroy, y sin `@csrf` Laravel la rechazará con el error **419 Page Expired**.

08 Trabajo a desarrollar

Complete el CRUD. Como mínimo, su proyecto debe respetar la siguiente estructura de archivos:

```
estructura del proyecto Copiar

app/
├── Http/Controllers/ProductoController.php ← CRUD: 7 métodos
├── Models/Producto.php ← modelo (provisto)
database/
├── migrations/xxxx_create_productos_table.php
├── factories/ProductoFactory.php ← datos con fake()
├── seeders/ProductoSeeder.php ← mínimo 20 productos
resources/views/
├── layouts/app.blade.php ← layout base (de la P01)
├── productos/
│   ├── index.blade.php ← listado (OBLIGATORIO)
│   ├── create.blade.php ← registro (OBLIGATORIO)
│   ├── edit.blade.php ← edición (OBLIGATORIO)
│   ├── show.blade.php ← detalle (OBLIGATORIO)
│   └── _form.blade.php ← campos compartidos (sugerido)
```

Las vistas deben cumplir, como mínimo, con los siguientes requisitos:

VISTA / ELEMENTO	REQUISITO MÍNIMO
Listado (index)	Tabla con nombre, SKU, categoría, precio (Bs, 2 decimales), stock y acciones (ver, editar, eliminar). Botón Nuevo producto y paginación con <code>\$productos->links()</code> .
Estado vacío	Mensaje cuando no existen productos, usando <code>@forelse</code> / <code>@empty</code> .
Registro (create)	Todos los campos de la tabla; <code>select</code> para la categoría y checkbox para destacado.
Edición (edit)	Mismo formulario con los datos actuales cargados; reutilice los campos (parcial <code>_form</code> o componente Blade).
Errores de validación	Cada campo conserva lo escrito con <code>old()</code> y muestra su error debajo con <code>@error</code> .
Detalle (show)	Todos los datos del producto, imagen si existe, badges de destacado / agotado (stock = 0) y enlaces a editar y volver.
Mensajes flash	Alerta visible tras crear, actualizar o eliminar, leyendo <code>session('success')</code> en el layout.
Eliminación	Formulario con <code>@method('DELETE')</code> y confirmación previa.

Requisitos técnicos:

- Rutas con nombre mediante el helper `route()`; no escriba URLs a mano.
- Route Model Binding en `show`, `edit`, `update` y `destroy`.
- Validación de todos los campos en `store` y `update`; nunca use `$request->all()` sin validar.
- Formularios protegidos con `@csrf` y method spoofing con `@method`.
- Interfaz con Blade y Tailwind CSS v4, responsiva, reutilizando el layout base.

09 Secuencia de commits sugerida

Se valora el historial, no solo el resultado final. Trabaje con commits atómicos:

#	MENSAJE DE COMMIT	CONTENIDO
1	chore: conexión a PostgreSQL	Configuración de <code>.env</code> verificada con <code>migrate</code>
2	feat: migración y modelo Producto	Tabla productos según la especificación
3	feat: factory y seeder	20 o más productos de prueba
4	feat: rutas resource y controlador	<code>Route::resource + ProductoController</code>
5	feat: listado de productos	<code>index</code> con paginación y estado vacío
6	feat: registro de productos	<code>create + store</code> con validación
7	feat: edición de productos	<code>edit + update</code> con SKU único
8	feat: detalle y eliminación	<code>show + destroy</code> con confirmación
9	style: mensajes flash y Tailwind	Alertas, badges y responsividad
10	docs: README	Documentación del repositorio

10 Entregables y revisión

- Repositorio en GitHub llamado `INF560-P02-Apellido` (reemplace Apellido por el suyo).
- Historial con commits atómicos (mínimo 8).
- Un archivo `README.md` con: nombre, descripción, pasos de instalación (`composer install`, `npm install`, copiar `.env.example`, `php artisan key:generate`, configurar la base de datos, `php artisan migrate --seed`) y capturas de las cuatro vistas.
- El enlace del repositorio, entregado mediante `virtual.uatf.edu.bo` en el apartado Práctica 2.

REVISIÓN EN CLASE · VIERNES 2 DE OCTUBRE

La revisión de esta práctica se realizará **durante la clase del día viernes 2 de octubre**. Cada estudiante presentará su proyecto funcionando en su propio equipo y deberá:

1. Ejecutar `php artisan migrate:fresh --seed` y mostrar el listado paginado con los datos de prueba.
2. Demostrar en vivo el registro, la edición y la eliminación de un producto, incluidos los mensajes de validación.
3. Explicar su código y responder las preguntas del docente sobre el flujo del CRUD.

Llegue con PostgreSQL, Vite y el servidor de Laravel listos para ejecutarse: el tiempo de revisión por estudiante es limitado.

ADVERTENCIA

No suba el archivo `.env` (contiene la contraseña de su base de datos) ni las carpetas `vendor/` y `node_modules/`: ya están excluidos en el `.gitignore` que Laravel trae por defecto.

11 Rúbrica de evaluación

CRITERIO	PTS
Migración según la especificación, modelo y seeder con 20 o más productos	15
Listado: tabla con paginación, formato de precio, acciones y estado vacío	15
Registro (create / store) con validación, <code>old()</code> y errores por campo	20
Edición (edit / update) con validación y SKU único ignorando el propio registro	15
Detalle (show) y eliminación (destroy) con confirmación	10
Interfaz: layout reutilizado, Tailwind v4 responsivo y mensajes flash	10
Commits atómicos (mínimo 8) y README completo con capturas	5
Defensa en la revisión del viernes 2 de octubre: explica su código y responde preguntas	10
Total	100
<i>Bonus: subida de imagen con Storage en lugar de URL, funcionando en registro y edición</i>	+10

12 Bonus — Subida de imagen (opcional)

En lugar de pedir la URL de la imagen, permita que el usuario **suba un archivo** desde su computadora. Guárdelo en el disco `public` y almacene en la columna `imagen` solo la ruta relativa.

terminal

Copiar

```
# Enlaza storage/app/public con public/storage (una sola vez)
php artisan storage:link
```

en su controlador (store / update)

Copiar

```
// Regla de validación: 'imagen' => ['nullable', 'image', 'max:2048']
if ($request->hasFile('imagen')) {
    $datos['imagen'] = $request->file('imagen')->store('productos', 'public');
}
```

Recuerde agregar `enctype="multipart/form-data"` a la etiqueta `<form>` y mostrar la imagen con `asset('storage/' . $producto->imagen)`.